K-digital Training

영양 편의점

프로젝트 결과 보고서

팀 아이넷 | 이규남, 조혁준, 김종호, 박하영

목차

01 프로젝트 개요 ›

02 프로젝트 팀 구성 및 역할 ›

03 프로젝트 수행 절차 및 방법 ›

04 프로젝트 수행 결과 ›

05 자체 평가 의견 ›

01

프로젝트 개요

주제 선정 배경

사용자의 다양한 상황과 목적별 실제 편의점 질의 예시

예시 01 · 운동 후 / 예산	“ 운동 후 먹을 1만 원 미만의 음식을 추천해줘”
예시 02 · 초저예산 식단	“ 5000원으로 든든한 한 끼 추천해줘”
예시 03 · 야식 / 성분 케어	“ 내일 얼굴 안 붓게 짠 거 말고 야식 추천해줘”

주제 · 선정 배경 · 기획 의도

01 프로젝트 주제

맞춤형 영양 최적화 추천

자연어 입력(상황·예산) 기반의 편의점 맞춤형 영양 최적화 조합 추천 서비스

02 선정 배경

높은 탐색 비용 해소

건강 관리 수요 증가로 대조·조합 비용이 커짐에 따라, 자연어 질의만으로 예산 맞춤 최적 식품을 도출하는 서비스 필요성 대두

03 기획 의도

자동화 및 선택 피로 감소

영양 판독/계산을 자동화하여 선택 피로를 줄이고 정량적 영양 데이터(DB) 기반 최적 식단 조합 제공

유사 서비스와의 차별화

기존 서비스에 없는 '목적 + 예산 + 정확한 영양 합계'

서비스	핵심 기능	한계	★ 우리의 차이
편의점 자체앱 (포켓CU 등)	자사 상품·행사·재고 조회	목적별 조합 추천과 영양 합산이 없음	목적·예산에 맞는 조합과 영양 합계 제공
범용 LLM 챗봇	자연어로 조합 추천	실제 제품·가격·영양을 추정(환각)	LLM은 해석만, 수치는 DB·코드가 계산
펴늬 (편의점 행사 제품 정보)	여러 편의점 행사 비교	행사 중심, 영양 기반 추천 없음	영양 목표 기반 추천
식단 기록 앱	직접 검색 후 기록	구매 전 선택은 지원하지 않음	메뉴 고르기 전에 바로 추천

프로젝트 내용

01 영양 DB 구축

- CU 앱 상품 상세 이미지 크롤링
- Vision LLM(Qwen2.5-VL)이 영양성분표를 JSON으로 파싱
- 열량·탄수화물·당류·단백질·지방·나트륨을 SQLite에 저장

02 요청 해석 (LLM)

- 한 문장 → 영양소 2개 + 방향 + 예산(JSON)
- A.X-4.0-Light QLoRA 파인튜닝
- parse_query 함수로 API에 제공

03 추천 · 서비스

- Python·SQL이 후보 필터링·랭킹·합계 계산
- API·예산 규칙, 화면
- LLM이 추천 이유를 작성

LLM 파인튜닝(QLoRA), 프롬프트 설계, Vision LLM, DB, 평가 설계를 하나의 서비스로 연결했습니다.

프로젝트 구조

① 데이터 구축

01
편의점 상품 상세
영양성분표 이미지

→

AI
Vision LLM
이미지 → JSON

→

DB
영양 DB
(SQLite)

⇢

수치 계산은
코드가 담당

↓

② 추천 서비스

IN
사용자 문장
+ 카테고리 선택

→

AI
Agent LLM (A.X)
문장 → 조건 JSON

→

CODE
Python · SQL
필터·랭킹·합계

→

OUT
Top 5 추천메뉴
최종 제공

활용 장비 및 재료

언어 · DB	Python, SQLite
데이터 수집	Playwright·Selenium으로 CU 앱 상품 상세 이미지 크롤링
Vision LLM	영양성분표 이미지 (Qwen2.5-VL-7B-Instruct) → JSON 파싱, 영양 DB 구축
Agent LLM	A.X-4.0-Light(SKT, 7B) · QLoRA 4bit
장비	학습 서버 RTX 4090 / 시연·재현 노트북 RTX 4060 8GB(WSL)
협업 도구	GitHub(브랜치 → PR → 팀장 승인), Git LFS(어댑터), 공유 문서 docs/interface.md
재료(데이터)	학습 문장 2,820개 · dev 216개 · test 240개 / CU 완제품 영양 DB

활용 방안 및 기대 효과

01. APPLICATION

활용 방안

- 편의점 앱·챗봇 연동
  ("운동 후 단백질" 등 대화형 조합 추천)
- GS25 등 타 편의점 서비스로의 유연한 채널 확장

02. USER BENEFIT

사용자 효과

- 영양성분표 확인 및 가격 계산 시간 대폭 절감
- 설정된 예산 한도 내 정확한 영양성분 합계 실시간 확인

03. BUSINESS

비즈니스 효과

- 리뷰 평점 데이터 기반 트렌디한 인기 꿀조합 파악
- 행사 정보 연계 추천 및 고객 맞춤형 큐레이션 서비스 구축

04. TECH EFFECT

학습 · 기술 효과

- LLM 기반 메뉴·예산·영양소 자동 추출
- 조건 충족 최적 Top 5 추천 메뉴 조합 알고리즘 제공

02

프로젝트 팀 구성 및 역할

프로젝트 팀 구성 및 역할

훈련생	역할	담당 업무
박하영	자연어처리, Agent LLM	요청 해석 데이터 설계 · 정제, A.X QLoRA 파인튜닝 · 평가, parse_query 함수 · 입력 안전장치
김종호	모델 비교평가, API	모델 비교 평가, test 문장 240개 작성, 가격(예산) 필터, FastAPI 연결
이규남	Vision LLM, DB 구성, 추천 조합 필터	이미지 크롤링, Vison LLM, 후보 추출 · 랭킹 엔진, 검증 · 합산, 팀 공유 문서 관리
조혁준	UI/UX, 양자화	화면(UI)과 안내문구, CU 데이터 수집 · 매칭, 모델 비교 및 양자화

03

프로젝트 수행 절차 및 방법

프로젝트 수행 절차 및 방법

STEP 01

사전 기획

- 문제·근거 확인
- MVP·성공 기준
- DB·JSON 설계

기획보고서
범위 동결

STEP 02

데이터 구축

- 이미지 크롤링
- Vision LLM 파싱

영양 DB

STEP 03

요청 해석 모델링

- 성분 1→2개 시도
- 짝 규칙·데이터 정리
- QLoRA 파인튜닝

Validation
dev 86.1%

STEP 04

서비스 통합

- parse_query와 API 연결
- 추천 엔진·화면 통합

통합 MVP

STEP 05

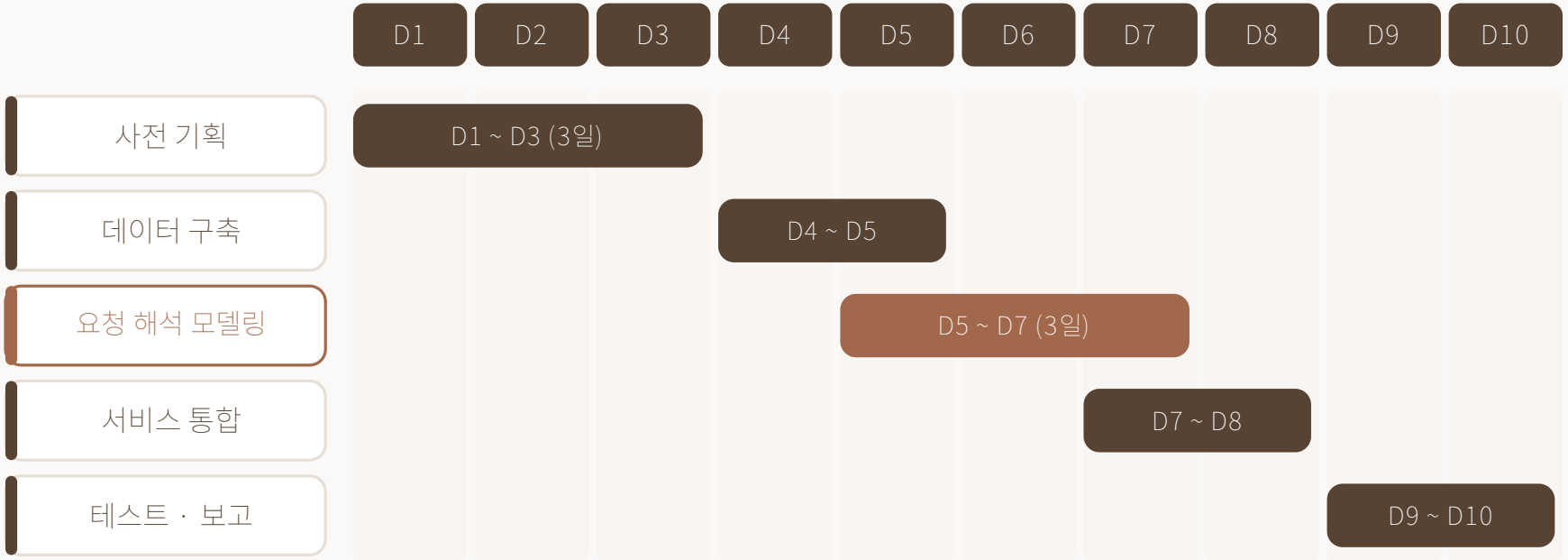
테스트 · 보고

- test 240문장 1회 평가
- 오류 수정·발표

Final Result
test 85.8%

프로젝트 일정

총 개발기간 10일 (계획 기준)



04

프로젝트 수행 결과

데이터 준비 - 요구 사항 분석

최종 DB(cu_products) 스키마가 요구하는 3대 핵심 데이터 축

1. 식별 및 상품 메타
상품명, 상품코드, 카테고리, 상품 이미지 url

편의점 매장 진열 품목 식별 및 추천 UI 카드 노출용

2. 상거래·추천 제약 (공공DB 부재)
가격, 이벤트 타입(1+1, 2+1, NONE)

예산 내 가성비 조합' 계산을 위한 필수 필터 조건

3. 정밀 6대 영양소 (VLM 추출 대상)
열량, 단백질, 나트륨, 탄수화물, 당류, 지방

'고단백/저당/저나트륨' 목표 달성용 연속형 수치

[한계] 식약처 공공 DB 사용 불가 사유

1. 핵심 메타데이터(가격/행사) 부재

영양 규격 중심 데이터셋으로 필수인 판매가 및 1+1/2+1 프로모션 정보 전무

2. 실시간 편의점 유통 데이터 미반영

1~2주 단위 빠른 신상품 및 단종 주기를 못 따라가 매장 상품 매칭률 급감

3. 상품명 불일치로 JOIN 불가

정식 제조 품목명과 소비자가 인지하는 편의점 유통명이 달라 대량 결측 발생

[해결] 실물 라벨 크롤링 + Vision-LLM 도입

1. 100% 실판매 데이터 및 이미지 확보

CU 웹/앱 크롤링으로 최신 상품명, 가격, 행사 정보와 고화질 영양 라벨 수집

2. 전통적 OCR 한계 극복

2D 좌표 간섭, 줄바꿈 오인식, 컬럼 밀림을 문맥 이해형 Vision-LLM으로 해결

3. Qwen2.5-VL 기반 구조화

실물 라벨의 6대 영양성분을 정밀 판독하여 최종 DB 스키마 규격으로 직접 적재

데이터 준비 - Vision LLM 엔지니어링

Vision-LLM 데이터 추출 & 엔지니어링 파이프라인 통합

1. VLM 모델 선정 & 하드웨어 가속

• Qwen2.5-VL-7B-Instruct

동적 해상도 기반 비정형 영양성분표 판독 최적화

• GPU 최적화 & 초고속 추론

RTX 4060 (NF4 4-bit) ~ RTX 4090 (bf16, 장당 0.5초대 처리)

• 초고해상도 이미지 DOS 방어

초고해상도 상세페이지 통이미지 처리 시 발생하는 PIL 크기 제한 오류 방어

2. 프롬프트 & 파이프라인 제약 설계

• 엄격한 단위 제약 & Few-shot

[kcal, mg, g] 단위 한정 및 명확한 올바른 예시 명시

• 정형 JSON 스키마 강제

마크다운/인사말 제외, 파이썬 즉시 dict 파싱 가능 출력

• 후처리 클렌징 안전장치

정규식 기반 숫자/단위 분리, 오인식 발생 시 None 처리

데이터 준비 - Text-to-SQL 기반 RAG

최종 SQLite DDL 스키마 (cu_products)

최종 SQLite DDL 스키마Table: cu_products

```
CREATE TABLE cu_products (
item_cd TEXT PRIMARY KEY, -- 바코드 (상품 ID)
category TEXT NOT NULL, -- '간편식사' 등
product_name TEXT NOT NULL, -- 상품명
price INTEGER CHECK(price > 0), -- 판매가
event_type TEXT DEFAULT 'NONE', -- 1+1, 2+1 등
-- Vision-LLM(Qwen2.5-VL) 추출 6대 영양성분
calories REAL CHECK(calories >= 0), -- 열량(kcal)
protein REAL CHECK(protein >= 0), -- 단백질(g)
sodium REAL CHECK(sodium >= 0), -- 나트륨(mg)
carbs REAL CHECK(carbs >= 0), -- 탄수화물(g)
sugar REAL CHECK(sugar >= 0), -- 당류(g)
fat REAL CHECK(fat >= 0), -- 지방(g)
thumbnail_url TEXT, price_date TEXT NOT NULL
);
CREATE INDEX idx_category_price ON cu_products(category,
price);
```

✓ 무결성 제약: CHECK 8건 반영 ⚡ 복합 인덱스 조회 최적화

카테고리별 구축 및 추천 필터링 현황

카테고리별 구축 및 추천 필터링 현황 기준: 실물 라벨 유효성 검증

카테고리	전체 수집	추천 대상	추천 제외
식품	768	746	22
과자류	627	627	0
음료	411	411	0
간편식사	199	199	0
아이스크림	109	109	0
즉석조리	40	38	2
합계	2,154	2,130	24

✓ 실물 라벨 유효성 검증 완료
전체 수집 2,154건 중 2,130건(98.9%) 추천 대상 확정

기능 구현 - 추천 흐름

입력 예시 “운동 끝나고 5천원으로 먹을 거 추천해줘” + 카테고리 선택: 간편식사, 음료

1 자연어 처리

LLM

영양 조건 2개 추출
(QLoRA 파인튜닝, JSON)
예산은 규칙으로 추출

결과
단백질 high
지방 low
예산 5,000원

2 가격 필터

예산 → 검색 조건

예산이 있으면
단품 가격 상한 조건 추가
없으면 가격 제한 없음

결과
price ≤ 5,000

3 DB 조회

SQLite

카테고리별 후보 검색
추천 제외 상품 24개는
검색에서 빠짐

결과
간편식사 158개
음료 404개

4 Top 5 필터

Python 필터 엔진

카테고리별 1개씩 조합
합계 ≤ 예산
영양 0.7 + 가격 0.3 점수
중복 상품 제거

결과
63,832개 조합
→ Top 5
1위 82.3점 · 4,900원

5 화면 표시

웹 추천 카드

순위별 상품, 합계 가격
영양 점수, 행사 정보
(1+1 · 2+1) 표시

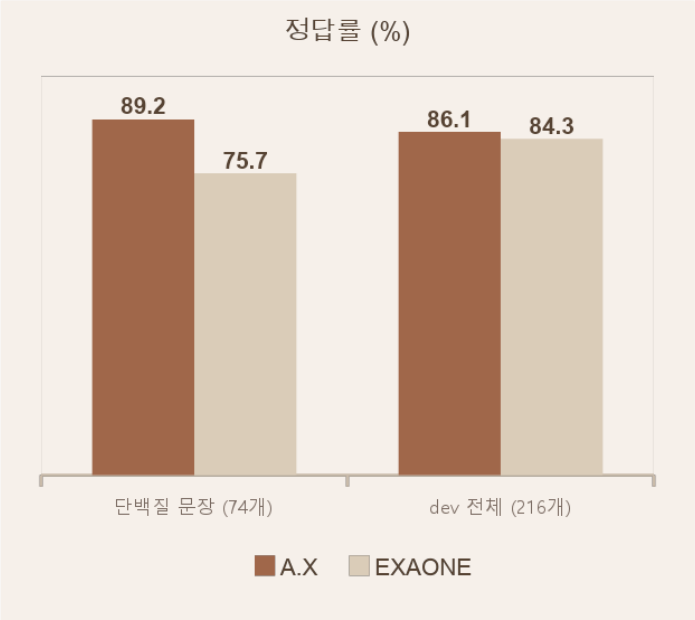
결과
추천 카드 5개

모델 비교 평가 (A.X vs EXAONE)

86.1%	84.3%	15 : 11	19
A.X · 186 / 216	EXAONE · 182 / 216	한 모델만 맞힌 문장 (A.X : EXAONE)	두 모델 모두 틀림 (16개는 같은 오답)

구분	입력 문장	정답	A.X	EXAONE
A.X만	아기 수유 중인데 왜 이렇게 배가 고프지? 뭐 먹을까?	단백질↑ 지방↓	✔ 단백질↑ 지방↓	✘ 열량↑ 탄수↑
A.X만	성장기 아이 간식	단백질↑ 지방↓	✔ 단백질↑ 지방↓	✘ 나트륨↓ 지방↓
A.X만	신검 코앞 ㄷㄷ 가벼운거 머먹지	열량↓ 지방↓	✔ 열량↓ 지방↓	✘ 탄수↓ 단백질↑
EXAONE만	회식하고 나서 몸이 무거운데 뭐 먹지	나트륨↓ 지방↓	✘ 단백질↑ 지방↓	✔ 나트륨↓ 지방↓
EXAONE만	면먹고 배 터질듯 ㅜㅜ 오늘 딴거 머먹지	탄수↓ 단백질↑	✘ 열량↓ 지방↓	✔ 탄수↓ 단백질↑
EXAONE만	1,500원에 우는 아이 그치게 할 만한 게 뭘까요?	당류↑ 열량↑	✘ 나트륨↓ 지방↓	✔ 당류↑ 열량↑
둘 다 틀림	오늘 제주 올레길 걷는데 중간에 뭐 먹으면 힘이 날까?	당류↑ 열량↑	✘ 탄수↑ 열량↑	✘ 탄수↑ 열량↑
둘 다 틀림	모유 수유로 늘어난 식사량을 채우고 싶습니다.	단백질↑ 지방↓	✘ 열량↑ 탄수↑	✘ 열량↑ 탄수↑
둘 다 틀림	아이 간식 3천원 안쪽으로 골라줘	단백질↑ 지방↓	✘ 나트륨↓ 지방↓	✘ 나트륨↓ 지방↓

모델 A.X 선정 이유



단백질 정답 문장 74개	A.X	EXAONE
완전 정답	66 (89.2%)	56 (75.7%)
한 모델만 맞힘	13	3
단백질을 빠뜨린 답	7	18

EXAONE이 틀린 예

- " 성장기 아이 간식" → 나트륨↓ 지방↓
- " 수유 중 배고픔" → 열량↑ 탄수↑

정답은 둘 다 단백질↑ 지방↓, A.X는 맞힘

선정 결론

단백질은 운동·성장·수유 등 핵심 추천 상황의 주 성분이라 A.X를 유지

모델 선정 (A.X-4.0-Light)

기반 모델 : Qwen 2.5-7B

1	한국어 특화	한국어 벤치마크 3종 모두 기반 모델보다 높음 (KMMLU +14.6점). 영어 성능(MMLU)은 그대로 유지
2	가벼운 7B 크기	4bit로 줄이면 약 4.95GiB → RTX 4060 8GB 노트북에서 시연 가능, 문장당 약 3.3초
3	Apache 2.0 라이선스	상업적 이용·수정·배포 가능. 서비스로 확장할 때 라이선스 부담이 적음
4	표준 구조 (Qwen2)	transformers 기본 지원 → QLoRA(peft) 파인튜닝을 바로 적용.

학습 데이터 준비

QLoRA 학습 문장 2,820개 (v3)

2,820

전체 데이터 (94상황 × 30문장)

2,550 + 270

학습 + 검증

학습 데이터 예시 (문장 → 정답)

구어체 — 나트륨↓ 지방↓

"밤에 배고파서 잠이 안 와 뭐 먹을까"

줄임말·오타 — 단백질↑ 지방↓

" 감량중인데 근육빠지면 안되는데ㅠ"

예산 포함 — 탄수화물↑ 열량↑

" 지갑에 6500원 있는데 내일 달리기 대비해서 힘 나는 걸로 뭐 살까?"

성분 직접 언급 — 탄수화물↓ 단백질↑

" 프로필 찍기 전까지 탄수화물 낮은 걸로만 먹고 싶어요."

데이터 구성

- 문장 → 정답 JSON (영양소 2개 + 높음·낮음)
- 성분 6개: 열량·단백질·나트륨·탄수화물·당류·지방
- 말투 7가지: 구어체 · 줄임말 · 예산 포함 등
- dev·test는 학습 문장과 상황이 겹치지 않음

데이터 정리 (v1→v3)

- 팀 라벨 기준 확정, '단짠'(나트륨↑) 추가
- 기준과 어긋난 69문장 수정

학습 설정 (QLoRA)

- 4bit NF4 + LoRA r16·α32·dropout 0.05
- 학습률: 2e-4, 유효 배치: 16
- 최대 10 epoch, EarlyStopping: patience 3
- best = epoch 1 (검증 86.7%), epoch 4에서 종료
- RTX 4090, 약 56분

성능 개선 과정

1단계

성분 1개 추출

성분 1개 + 방향
학습 예시 9개

82.9%

성분 1개는 충분히 맞힘

2단계

성분 2개로 확장

성분 2개 + 방향
(운동 후 → 고단백, 저지방)

14.4%

둘째 성분 22.7%만 맞힘

3단계

짝 규칙표로 통일

6개 중에서만 고르도록
규칙 예시 10개 추가

44.0%

둘째 성분 52.3%로 상승

4단계

데이터 v3 정리

라벨 기준 확정
69개 문장 수정

43.5%

44% 부근에서 정체

5단계

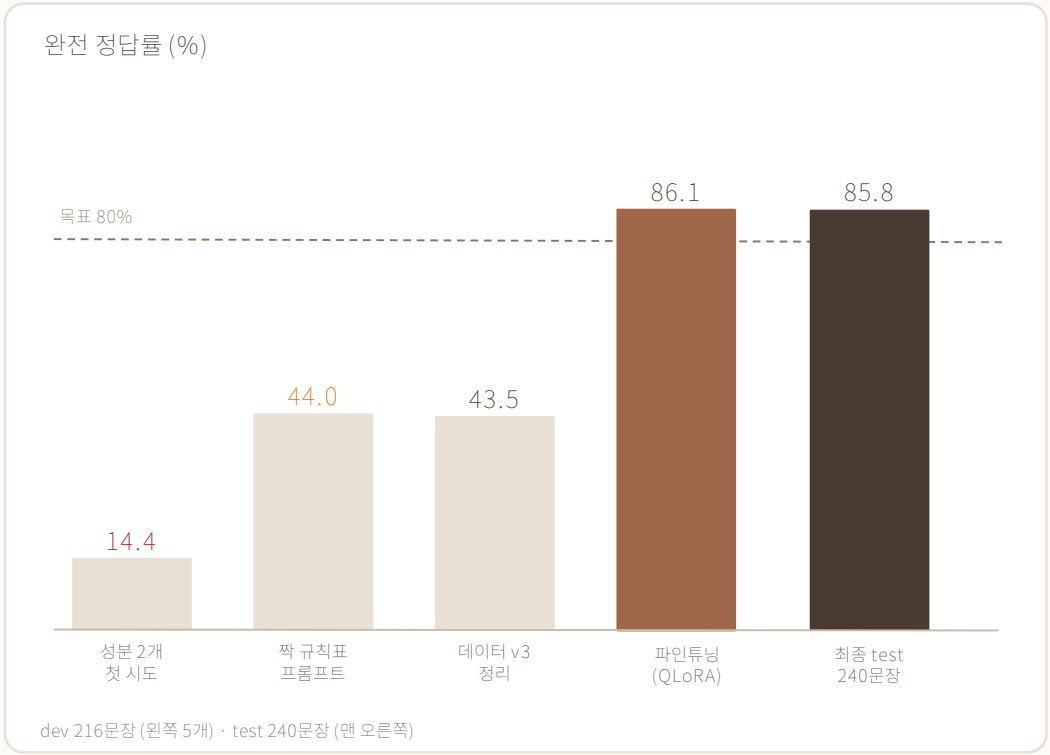
QLoRA 파인튜닝

학습 데이터 2,820쌍으로
모델 자체를 학습

86.1%

dev 기준 · 한계 돌파

성능 지표 변화



최종 결과

86.1% / 85.8%

dev / test · 목표 80% 달성

프롬프트 단계

14.4% → 44.0%. 예시를 바꿔도 44% 부근에서 정체

파인튜닝

+36.6%p. 처음 보는 test에서도 85.8%로 유지

추가 예외 규칙

무관한 문장(인사/날씨 등)은 빈 값으로 답해 잡담 추천 방어 `{"nutrients":[]}`

최종 성능 개선

재학습 없이, 규칙 2단계로 해결

① 문제

모델은 어떤 문장이든 영양소 2개를 고르도록 학습되어, 잡담에도 영양소를 지어냄

“ 오늘 날씨 어때”	열량↓ 지방↓
“ 심심해”	단백질↑ 지방↓
“ 뭐해 지금”	단백질↑ 지방↓

→ 먹을 것과 관계없는 말에도 상품을 추천하는 문제

② 해결 : 2단계로 거르기

1단계 · 규칙 검사

인사·요청어만 있으면 바로 차단
모델 호출 없음 (예: 안녕하세요)

2단계 · 시스템 프롬프트

학습 프롬프트 끝에 규칙 추가
모델은 다시 학습하지 않음

관계없는 문장 규칙:
사용자 문장이 먹을 것, 식사, 건강, 몸 상태, 상황과 관계 없으면 (인사, 잡담, 날씨, 감사 인사, 다른 주제 질문 등) 영양소를 고르지 말고 `{"nutrients": []}` 로만 답하세요.

“ 오늘 날씨 어때” → `{"nutrients": []}` → 안내 문구

③ 결과 · 잡담 차단

6 / 6

수정 전에는 1 / 6만 걸러짐
추가 2문장 포함 8 / 8

인사·요청어
13 / 13 차단
정상 문장 정확도 (dev 216)
86.1% → 85.6%
1문장을 관계없음으로 잘못 판단

가격(예산) 필터

영양소는 모델, 가격은 규칙 코드가 담당

test 예산 포함 58문장 정확도

53.4%

파인튜닝 전 (원본 4bit)

86.2%

파인튜닝 후

예산 추출 규칙 (budget_filter)

문장	파인튜닝 전	파인튜닝 후
예산은 6천이야	없음	6,000
4천원 대신 3천원으로	4,000	3,000
만원 넘게 써도 돼 (상한 아님)	10,000	없음
나트륨 2천 이하로 (금액 아님)	없음	없음
천원짜리 지폐 두 장	1,000	2,000

DB 조회 (후보 검색)

화면에서 고른 카테고리마다 SQLite에서 후보 상품을 꺼낸다. 예산이 있으면 단품 가격 상한을 함께 건다

실행되는 SQL (카테고리마다 1번)

```
SELECT item_cd, category, product_name,
       price, event_type, calories, protein,
       sodium, carbs, sugar, fat, thumbnail_url
FROM cu_products
WHERE category = ?
  AND exclude_reason IS NULL
  AND price <= ?      -- 예산이 있을 때만
ORDER BY price, item_cd
```

✓ 읽기 전용 연결 (mode=ro) ✓ 값은 ? 파라미터로 전달

입력 → 출력 (예시)

입력
카테고리 간편식사, 음료
예산 5,000원

출력 : 카테고리별 후보 목록 (가격 오름차순)
간편식사 158개 / 199개
음료 404개 / 411개
상품마다 바코드, 이름, 가격, 행사, 영양소 6개, 이미지

입력 검사
카테고리 1~3개, 예산은 양의 정수
없는 카테고리, 중복, 잘못된 예산은 오류로 거부

추천 제외
제외 표시된 24개는 검색 안 함
소스·양념·조미료와 식재료. DB에는 남김

역할 분리
영양 조건은 쿼리에 넣지 않음
순위는 다음 단계인 Top 5 필터가 점수로 계산

Top 5 필터 (점수 계산)

조합마다 영양 점수와 가격 점수를 7 : 3으로 합쳐 총점을 매긴다. 계산은 LLM 없이 Python 코드가 DB 값으로 한다

점수 규칙

1 영양 점수 상품마다

같은 카테고리 후보 안의 백분위 (0~100)

- high 조건은 그대로, low 조건은 100 − 백분위
- 조건이 2개면 두 점수를 평균

2 가격 점수 조합마다

합계 ÷ 예산 × 100

- 예산에 가까울수록 높음
- 합계가 예산을 넘는 조합은 계산하지 않음

3 총점 조합마다

0.7 × 영양 점수 + 0.3 × 가격 점수

- 조합의 영양 점수 = 상품 영양 점수의 평균
- 예산이 없거나 카테고리가 1개면 영양 점수만 사용

계산 예시 : 1위 조합

조건 단백질 high · 지방 low | 예산 5,000원 | 간편식사 + 음료

간편식사 샌)급식대가저당치킨샌드
3,600원 단백질 26g · 지방 6g
영양 점수 90.2

음료 레쓰비)캔200ml
1,300원 단백질 1g · 지방 0g
영양 점수 60.9

영양 점수	(90.2 + 60.9) ÷ 2	75.5
가격 점수	4,900 ÷ 5,000 × 100	98.0

총점
0.7 × 75.5 + 0.3 × 98.0
82.3

Top 5 필터 (조합과 선정)

카테고리마다 1개씩 묶은 조합 중 예산 안의 것만 남기고, 총점 순으로 보면서 중복 상품을 건너뛰어 5개를 고른다

조합이 줄어드는 과정 (예시)

63,832 — 전체 조합
간편식사 158 × 음료 404

19,622 — 예산 안 조합
합계 ≤ 5,000원

36 — 총점 순으로 검사
31개는 이미 쓴 상품이 있어 건너뜀

5 — Top 5
같은 상품은 한 번만 사용

Top 5 결과 (예시)

순위	간편식사	음료	합계(원)	총점
1	샌)급식대가저당치킨샌드	레쓰비)캔200ml	4,900	82.3
2	샌)호밀햄치즈샌드	매일)엔요280ml	5,000	79.4
3	빅삼)정통돈까스삼각	매일)셀렉스프로핏코코넛	5,000	78.2
4	삼)떡갈비전주비빔삼각	유한)비컴플런부스터110ml	5,000	77.9
5	주)매콤양념족발밥바	풀무원)액티비아업딸기	5,000	76.5

정렬 순서
총점 → 영양 점수 → 낮은 합계
동점이면 영양 점수가 높은 조합이 먼저

중복 제거
Top 5 안에서 같은 상품은 한 번만
맛만 다른 상품(같은 카테고리·브랜드·가격, 이름 유사도 0.7 이상)도 제외

카테고리 3개
후보를 카테고리마다 80개로 축소
영양 상위 50개 + 나머지 중 저렴한 30개 (조합은 최대 80 × 80 × 80 = 약 51만 개)

서비스 준비 (양자화)

STEP 01 merge
어댑터를 원본에 층별로 더함
47초 · 약 10GB

→

STEP 02 GGUF
bf16 GGUF로 변환
48초 · 약 3.7GB

→

STEP 03 양자화
Q5_K_M으로 줄임,
약 5.2GB
1분 · 약 2.5GB

→

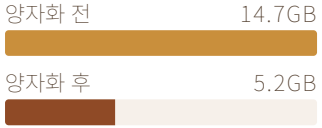
STEP 04 등록
Ollama 모델로 등록해 실행
수 초

01 모델 경량화

약 65% 감소

14.5GB + 어댑터에서
5.2GB 파일 하나로

필요한 파일 용량



02 4060 최적화

8GB 이내

모델 5.2GB +
실행 공간 약 1.5~2.5GB

메모리 8GB 중 사용량



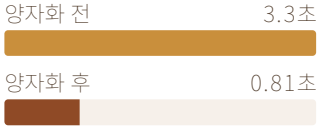
실행 공간(추정) 1.5~2.5GB

03 응답 속도 개선

약 4배

문장당 3.3초에서
0.81초로

문장당 응답 시간 (같은 4060)



04 정확도 유지

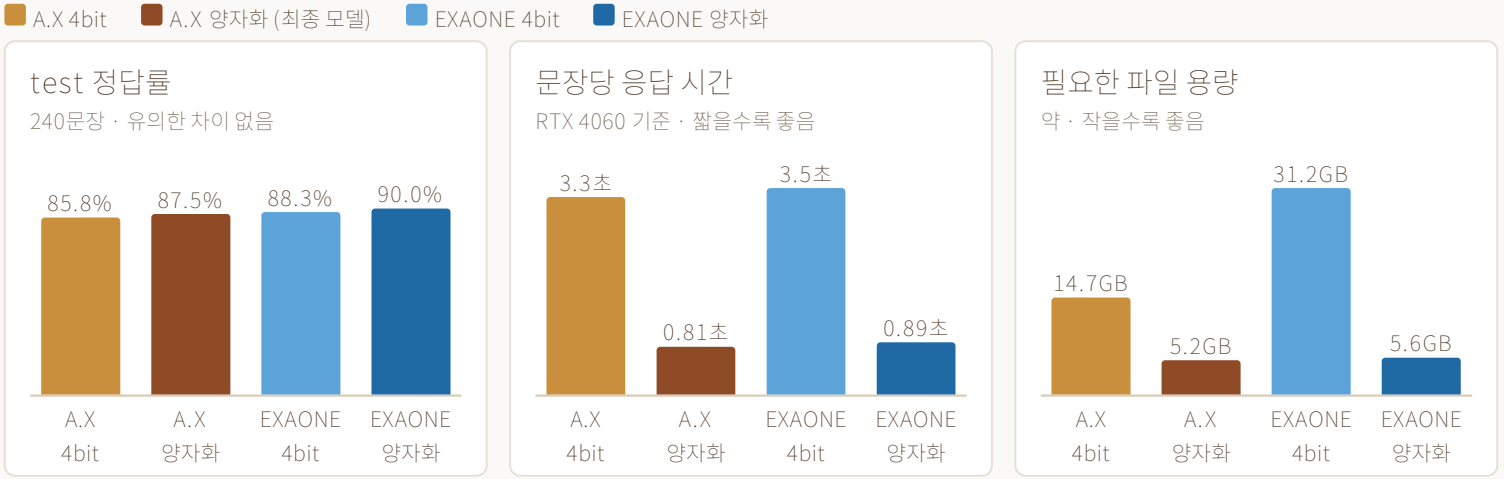
손실 없음

dev 186 → 186
test 206 → 210
문장별 예측 213/216 일치 · 형식 오류 0건

dev 정답 수 (216문장)

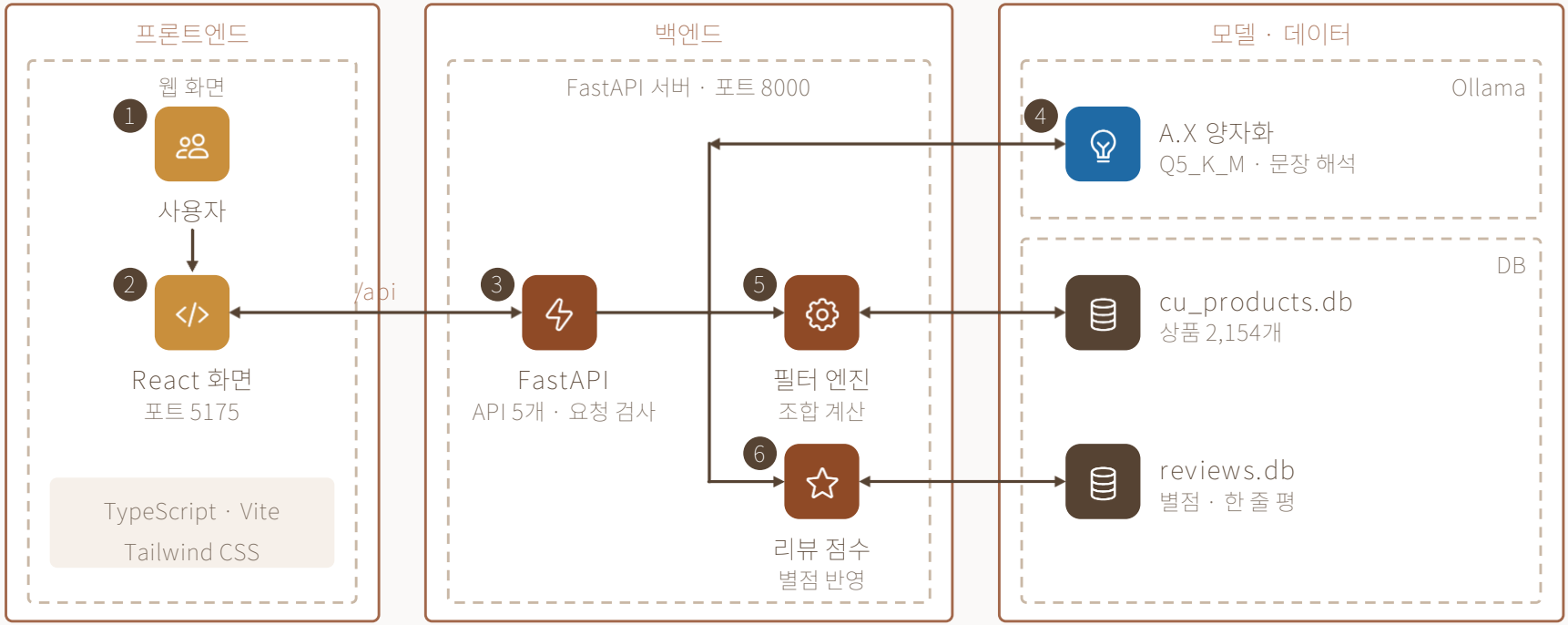


프로젝트 수행 결과 (최종 모델 비교 분석)



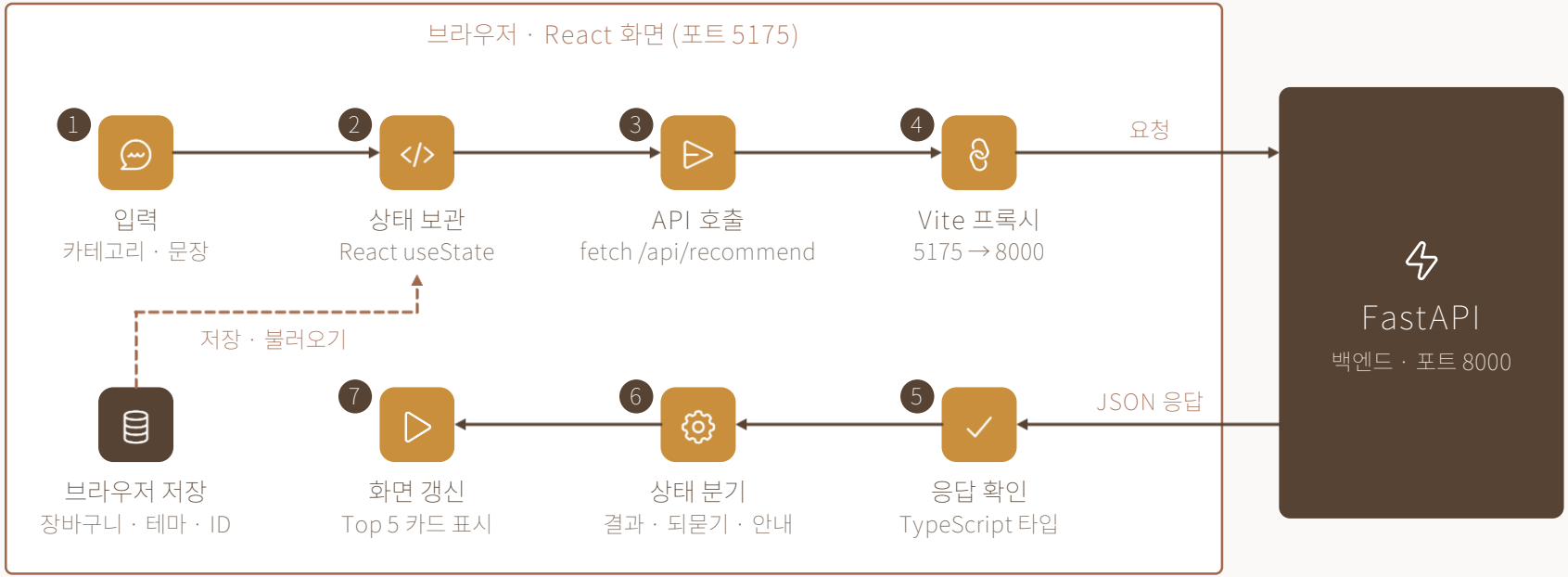
구분	4bit + 어댑터 (양자화 전)	양자화 GGUF (양자화 후)
어댑터	원본은 그대로 두고, 실행할 때마다 어댑터를 얹음	어댑터를 원본에 합쳐 파일 하나로 만듦
압축 방식	NF4 · 불러올 때마다 가중치를 4비트로 압축	Q5_K_M · 미리 약 5비트로 압축해 파일로 저장
실행 도구	Transformers · Python 코드로 모델을 불러와 실행	Ollama · 파일을 등록하면 명령 하나로 실행

시스템 아키텍처


프론트엔드
웹 화면
1
사용자
2
React 화면
포트 5175
TypeScript · Vite
Tailwind CSS
/api
백엔드
FastAPI 서버 · 포트 8000
3
FastAPI
API 5개 · 요청 검사
5
필터 엔진
조합 계산
6
리뷰 점수
별점 반영
모델 · 데이터
Ollama
4
A.X 양자화
Q5_K_M · 문장 해석
DB
cu_products.db
상품 2,154개
reviews.db
별점 · 한 줄 평

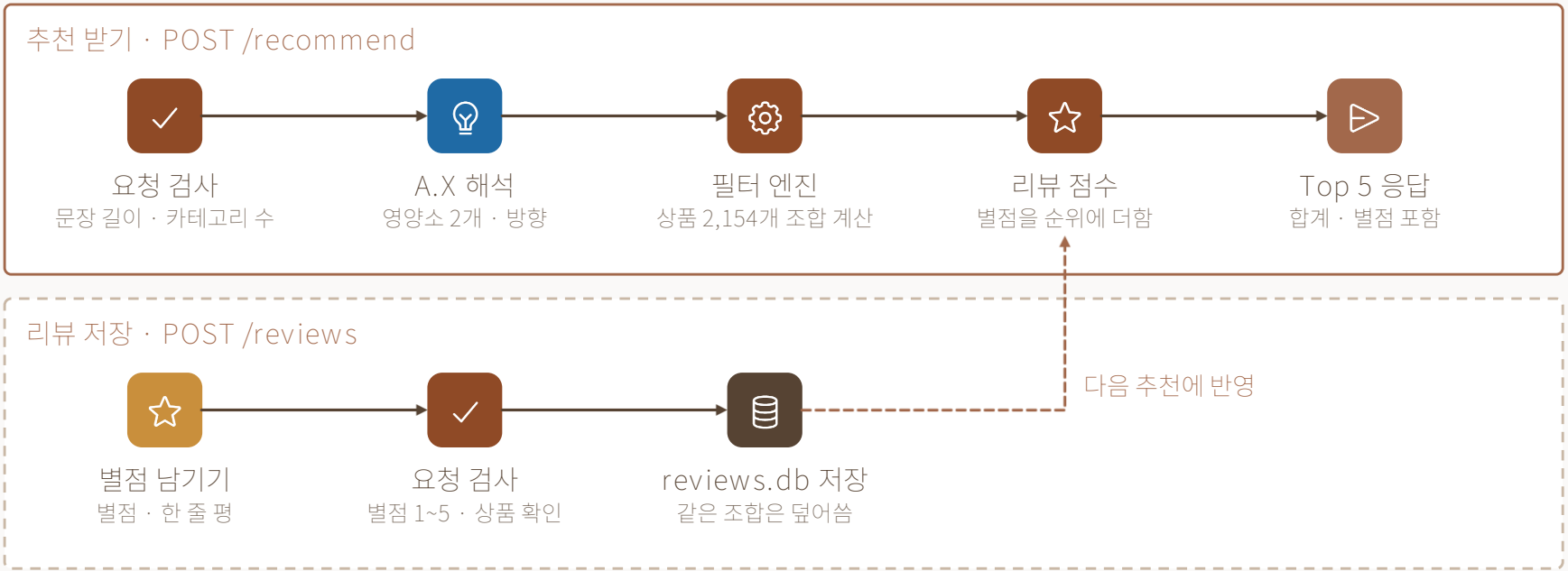
프론트엔드

React 18 화면 컴포넌트	TypeScript 응답 형식 검사	Vite /api를 서버로 전달	Tailwind CSS 스타일·반응형	CSS 변수 다크·화이트 전환	lucide-react 아이콘

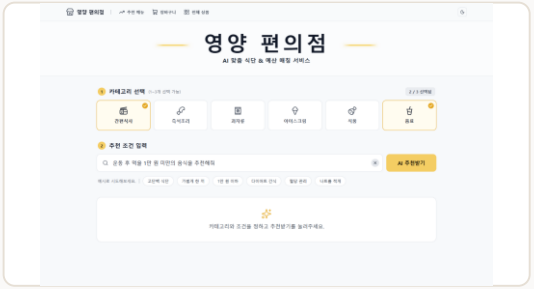


백엔드 · FastAPI (API 5개)

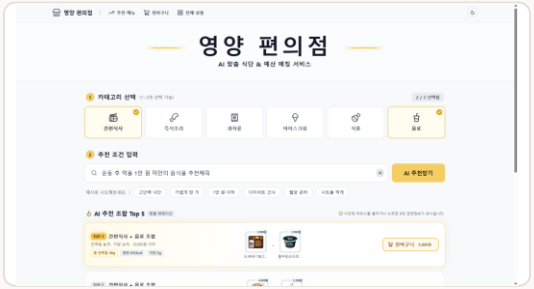
POST /recommend	POST /reviews	GET /categories	GET /products	GET /reviews
문장으로 Top 5 추천	별점·한 줄 평 저장	카테고리 목록	전체 상품 검색·정렬	저장된 별점 조회



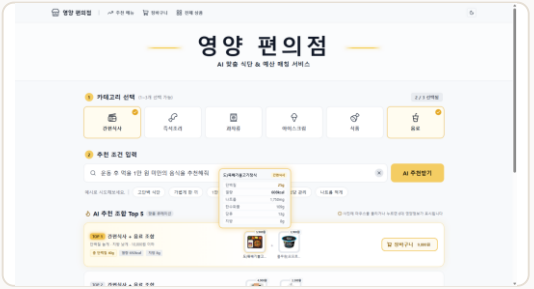
사용자 화면 흐름



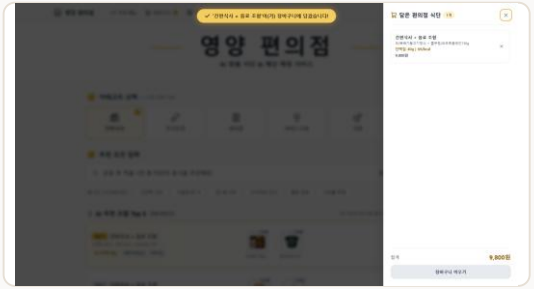
1. 조건 입력
카테고리를 고르고 평소 말투로 문장 입력



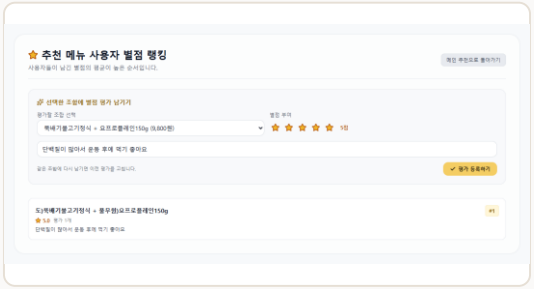
2. Top 5 결과
조합별 가격과 영양 합계 표시



3. 영양정보 확인
상품 사진을 누르면 6대 영양정보 표시



4. 장바구니 담기
조합을 담고 합계 금액 확인



5. 별점 남기기
추천 메뉴 탭에서 별점 · 한 줄 평, 랭킹 확인

흐름

조건 입력

→ Top 5 결과

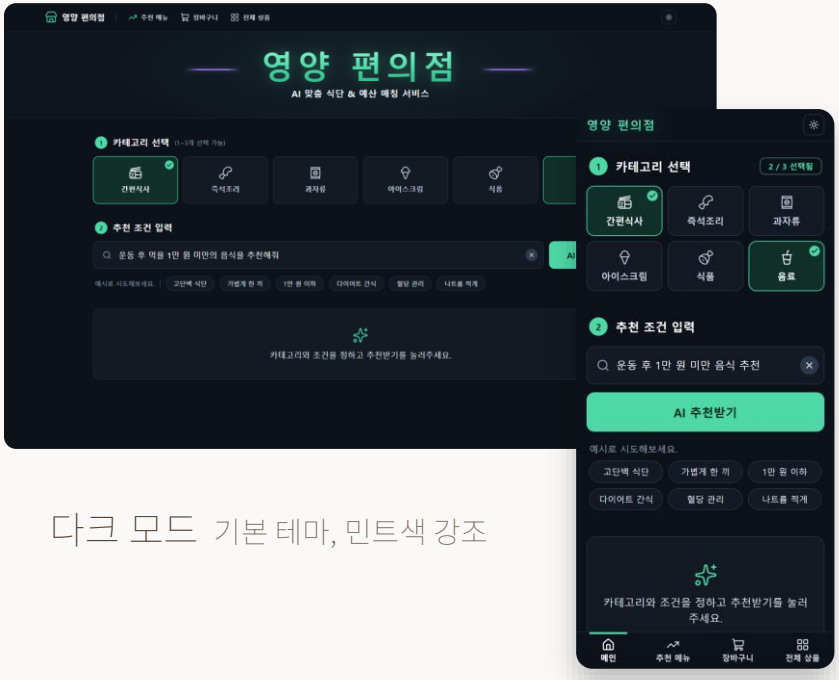
→ 영양정보 확인

→ 장바구니 담기

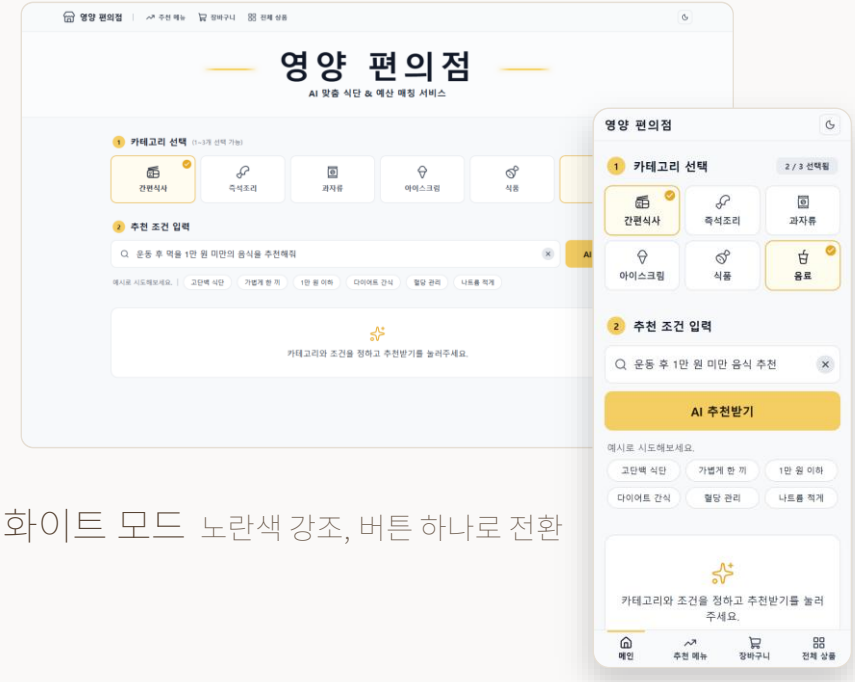
→ 별점 남기기

남긴 별점은 다음 추천 순위에 반영

웹 · 모바일 화면 구성



다크 모드 기본 테마, 민트색 강조



화이트 모드 노란색 강조, 버튼 하나로 전환

자체 평가 의견

잘한 점

- 서버 상태가 좋지 않은 환경에서도 측정 → 개선 순서를 지켜 완전 정답률을 14.4%에서 86.1%까지 끌어올림
- 자체 성공 기준 80%를 넘은 85.8%를 달성

아쉬운 점

- 시간 문제로 카테고리 선택까지 자연어로 처리하려던 계획을 구현하지 못하고, 화면에서 직접 고르는 방식으로 남김
- 같은 이유로 학습을 충분히 더 진행하지 못해, 모델 성능을 더 끌어올릴 여지가 남음

개선 계획

- 안정적인 서버 환경에서 카테고리도 문장 한 줄에서 함께 추출하도록 확장하고, 학습 데이터를 늘려 재학습
- CU 외에 다른 편의점으로 영양 DB를 넓히고, 알레르기·건강 목적 같은 조건도 추가

시연 영상



시연 시나리오

- 운동 후 먹을 1만원 미만의 음식을 추천해 줘
- 오늘 날씨 어때?
- 500원으로 든든한 한 끼 추천해줘
- 내일 얼굴 안 붓게 짠 거 말고 야식 추천해줘

Q&A

문의 및 추가 논의

감사합니다

팀 아이넷 | 영양편의점